Name: Kanchan Thakur B2
Roll no.: 24
Branch: CSE (DS) - B
Semester: 6th
NLP Lab 7
AIM: Unigram, Bigram and Trigram

Theory:

Unigram:
A unigram is the simplest form of an n-gram in Natural Language Processing, where each individual word in a sentence is treated as a separate unit. It ignores context and considers only single words independently. Unigrams are mainly used for basic tasks like word frequency analysis, where the importance of each word is evaluated without considering its neighboring words.

Bigram:
A bigram consists of a sequence of two consecutive words from a sentence. Unlike unigrams, bigrams capture a small amount of context by considering word pairs, which helps in understanding simple relationships between words. They are commonly used in applications like predictive text and language modeling, where the probability of a word depends on the previous word.

Trigram:
A trigram is a sequence of three consecutive words taken together from a sentence. It provides more contextual information than unigrams and bigrams by considering a larger window of words. Trigrams are useful in more advanced language models, where better context leads to improved accuracy in tasks like next-word prediction and sentence generation.

Code:

```
import nltk
import pandas as pd
import matplotlib.pyplot as plt
from nltk.util import ngrams
from nltk.tokenize import word_tokenize
from collections import Counter
from wordcloud import WordCloud

nltk.download('punkt')


import nltk
nltk.download('punkt_tab')

from nltk import tokenize
from nltk.tokenize import sent_tokenize, word_tokenize

sentences = sent_tokenize(text)
words = word_tokenize(text)
print("No. of Words in corpus: ",len(words))
```

Output:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
True
```

```
import kagglehub

# Download latest version

path =
kagglehub.dataset_download("lakshmi25npathi/imdb-dataset-of-50k-movie-revi
ews")

print("Path to dataset files:", path)
```

Output:

```
Using Colab cache for faster access to the 'imdb-dataset-of-50k-movie-reviews' dataset.
Path to dataset files: /kaggle/input/imdb-dataset-of-50k-movie-reviews
```

Code:

```
df = pd.read_csv(f"{path}/IMDB Dataset.csv")

text = " ".join(df['review'].head(100))
```

```
tokens = word_tokenize(text.lower())
print("Total Words:", len(tokens))
```

```
Total Words: 28043
```

Code:

```
bigram = list(ngrams(tokens,2))

trigrams = list(ngrams(tokens,3)

bigram_freq = Counter(bigram)

trigram_freq = Counter(trigrams)

print("Top Bigrams")

print(bigram_freq.most_common(10))

print("\nTop Trigrams")

print(trigram_freq.most_common(10))
```

Output:

```
Top Bigrams
[(('<', 'br'), 458), (('br', '/'), 458), (('/', '>'), 458), (('>', '<'), 229), (('of', 'the'), 152), (('in', 'the'), 117), ((',', 'and'), 101), (('.', 'the'), 91), ((',', 'bu

Top Trigrams
[(('<', 'br', '/'), 458), (('br', '/', '>'), 458), (('/', '>', '<'), 229), (('>', '<', 'br'), 229), (('/', '>', 'the'), 43), (('.', '<', 'br'), 30), (('/', '>', 'i'), 23), ((
```

Output in Text:

```
Top Bigrams
```

```
[(('<', 'br'), 458), (('br', '/'), 458), (('/', '>'), 458), (('>', '<'),
229), (('of', 'the'), 152), (('in', 'the'), 117), ((',', 'and'), 101),
(('.', 'the'), 91), ((',', 'but'), 82), (('it', "'s"), 81)]
```

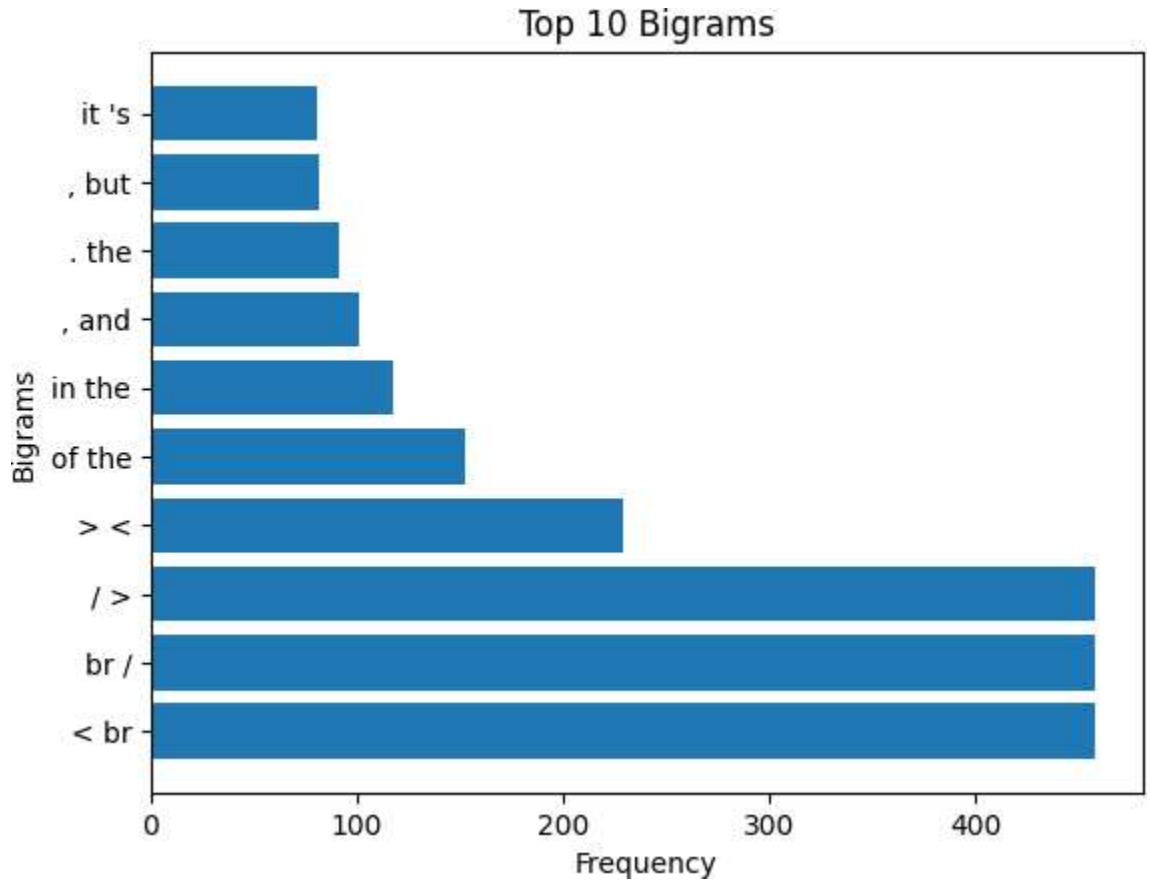
```
Top Trigrams
```

```
[(('<', 'br', '/'), 458), (('br', '/', '>'), 458), (('/', '>', '<'), 229),
(('>', '<', 'br'), 229), (('/', '>', 'the'), 43), (('.', '<', 'br'), 30),
(('/', '>', 'i'), 23), (('one', 'of', 'the'), 21), (('.', 'it', "'s"),
21), (('/', '>', 'this'), 15)]
```

Code:

```
top_bigrams=bigram_freq.most_common(10)

words=[" ".join(bg[0]) for bg in top_bigrams]

counts=[bg[1] for bg in top_bigrams]

plt.barh(words,counts)

plt.title("Top 10 Bigrams")

plt.xlabel("Frequency")

plt.ylabel("Bigrams")

plt.show()
```

Output:



Code:

```
bigram_text=[" ".join(bg) for bg in bigram_freq]

wordcloud=WordCloud(width=800,height=400,background_color='white').generat
e(" ".join(bigram_text))

plt.imshow(wordcloud,interpolation='bilinear')

plt.axis('off')

plt.title("Bigram Word Cloud")

plt.show()
```

Output:

Code:

```
def predict_next_word(word):

  candidates=[]

  for (w1,w2),freq in bigram_freq.items():

    if w1==word:

      candidates.append((w2,freq))

  candidates=sorted(candidates,key=lambda x:x[1],reverse=True)

  return candidates[:5]
```

```
predict_next_word("the")
```

Output:

```
[('movie', 60), ('film', 46), ('story', 26), ('first', 16), ('end', 16)]
```

Code:

```
def predict_next_word_trigram(w1,w2):

  results=[]

  for(a,b,c),freq in trigram_freq.items():

    if a==w1 and b==w2:

      results.append((c,freq))

  results=sorted(results,key=lambda x:x[1],reverse=True)

  return results[:5]
```

```
predict_next_word_trigram("i","love")
```

Output:

```
[('them', 1)]
```

Code:

```
trigrams=list(ngrams(tokens,3))

trigram_freq=Counter(trigrams)



print("Sample Trigrams:")

print(list(trigram_freq.most_common(10)))
```

Output:

```
Sample Trigrams:

[(('<', 'br', '/'), 458), (('br', '/', '>'), 458), (('/', '>', '<'), 229),
(('>', '<', 'br'), 229), (('/', '>', 'the'), 43), (('.', '<', 'br'), 30),
(('/', '>', 'i'), 23), (('one', 'of', 'the'), 21), (('.', 'it', "'s"),
21), (('/', '>', 'this'), 15)]
```

Code:

```
def autocomplete_senetnce(sentence):

  words=word_tokenize(sentence.lower())

  if len(words)<2:

    return "Enter atleast 2 words."

  w1,w2=words[-2],words[-1]

  predictions=[]

  for (a,b,c),freq in trigram_freq.items():

    if a==w1 and b==w2:

      predictions.append((c,freq))

  predictions=sorted(predictions,key=lambda x:x[1],reverse=True)

  return [word for word,freq in predictions[:5]]
```

```
sentence="this movie"
autocomplete_senetnce(sentence)
```

Output:

```
['is', ',', '.', 'was', 'and']
```

Code:

```
def generate_sentence(sentence):

  next_words=autocomplete_senetnce(sentence)

  results=[]

  for word in next_words:

    results.append(sentence + " " + word)

  return results
```

Code:

```
generate_sentence("this movie")
```

Output:

```
['this movie is',
 'this movie ,',
 'this movie .',
 'this movie was',
 'this movie and']
```

Code:

```
user_input=input("Enter a sentence: ")

predictions=generate_sentence(user_input)

print("\n Auto-Complete Sentences:")

for prediction in predictions:

  print(prediction)
```

Output:

```
Enter a sentence: the movie is

 Auto-Complete Sentences:
the movie is the
the movie is not
the movie is slower
the movie is watchable
the movie is pretty
```

Exercise: Apply N-gram analysis

Importing a text dataset of SMS from kaggle:

```
import kagglehub

path = kagglehub.dataset_download("uciml/sms-spam-collection-dataset")

print(path)



import os

files = os.listdir(path)

print(files)

df = pd.read_csv(os.path.join(path, files[0]), encoding='latin-1')

# dataset has 'v2' column = message text

text = " ".join(df['v2'].head(200)).lower()

tokens = word_tokenize(text)
```

Generate 4-gram sentences for the dataset
Code:

```
# 4 grams
four_grams = list(ngrams(tokens, 4))
print(four_grams[:10])
```

Output:

```
[('go', 'until', 'jurong', 'point'), ('until', 'jurong', 'point', ','),
('jurong', 'point', ',', 'crazy'), ('point', ',', 'crazy', '..'), (',',
'crazy', '..', 'available'), ('crazy', '..', 'available', 'only'), ('..',
'available', 'only', 'in'), ('available', 'only', 'in', 'bugis'), ('only',
'in', 'bugis', 'n'), ('in', 'bugis', 'n', 'great')]
```

Create a Trigram Word Cloud Visualization

Code:

```
trigrams = list(ngrams(tokens, 3))
trigram_freq = Counter(trigrams)

trigram_text = [" ".join(t) for t in trigram_freq]

wc = WordCloud(width=800, height=400, background_color =
'white').generate(" ".join(trigram_text))

plt.imshow(wc)
plt.axis("off")
plt.show()
```

Output:

Compare Bigram vs Trigram prediction accuracy

Code:

```
bigram_freq = Counter(ngrams(tokens, 2))
trigram_freq = Counter(ngrams(tokens, 3))

def predict_bigram(word):
    c = {k[1]: v for k, v in bigram_freq.items() if k[0] == word}
    return max(c, key=c.get) if c else None

def predict_trigram(w1, w2):
    c = {k[2]: v for k, v in trigram_freq.items() if k[0] == w1 and k[1]
== w2}
    return max(c, key=c.get) if c else None

test = tokens[:300]

b, t = 0, 0
num_predictions = len(test) - 2

for i in range(num_predictions):
    if predict_bigram(test[i]) == test[i+1]:
        b += 1
    if predict_trigram(test[i], test[i+1]) == test[i+2]:
        t += 1

print("Bigram Accuracy:", b/num_predictions)
print("Trigram Accuracy:", t/num_predictions)
```

Output:

```
Bigram Accuracy: 0.5369127516778524
Trigram Accuracy: 0.9697986577181208
```

Build a Sentence Auto Completion System

Code:

```
def autocomplete(text, num_predictions=5):
    w = word_tokenize(text.lower())
    predictions = []

    if len(w) >= 2:
        w1, w2 = w[-2], w[-1]
        trigram_candidates = []
        for (a, b, c), freq in trigram_freq.items():
            if a == w1 and b == w2:
                trigram_candidates.append((c, freq))
        trigram_candidates = sorted(trigram_candidates, key=lambda x:
x[1], reverse=True)

        for word, _ in trigram_candidates:
            if len(predictions) < num_predictions:
                predictions.append(text + " " + word)
            else:
                break

    if len(predictions) < num_predictions and len(w) >= 1:
        w1 = w[-1]
        bigram_candidates = []
        for (a, b), freq in bigram_freq.items():
            if a == w1:
                bigram_candidates.append((b, freq))
        bigram_candidates = sorted(bigram_candidates, key=lambda x: x[1],
reverse=True)
        for word, _ in bigram_candidates:
            if len(predictions) < num_predictions:
                if (text + " " + word) not in predictions:
                    predictions.append(text + " " + word)
            else:
                break

    if not predictions:
        return [text]
```

```
    return predictions

print("Autocomplete for 'free entry':")
for p in autocomplete("free entry"):
    print(p)
print("\nAutocomplete for 'call me':")
for p in autocomplete("call me"):
    print(p)
```

Output:

```
Autocomplete for 'free entry':
free entry in
free entry question

Autocomplete for 'call me':
call me now
call me once
call me just
call me .
call me ?
```

Conclusion: This practical successfully demonstrated N-gram analysis, including unigrams, bigrams, and trigrams. We visualized these with word clouds and compared bigram versus trigram prediction accuracy.

Finally, a functional sentence auto-completion system was built using N-gram models, showcasing their utility in natural language processing tasks.